

Sisteme de operare

Laborator 10

Semestrul I 2025-2026

Laborator 10

- POSIX threads
 - semafoare
 - monitoare

Semafoare POSIX

- cu nume (*named*) sau fara nume (*unnamed*)
- cele cu nume pot fi folosite de procese diferite (neinrudite), cele fara nume nu

Semafoare POSIX cu nume

- creare si initializare:

```
#include <semaphore.h>
sem_t *sem;

/* Create the semaphore and initialize it to 1 */
sem = sem_open("SEM", O_CREAT, 0666, 1);
```

- alte procese acceseaza semaforul cu ajutorul numelui **SEM**. (de fapt /SEM, sunt fisiere intr-un pseudo-sistem de fisiere */dev/shm*)
- obtinerea si eliberarea semaforului:

```
/* acquire the semaphore */
sem_wait(sem);

/* critical section */

/* release the semaphore */
sem_post(sem);
```

Semafoare POSIX fara nume

- creare si initializare:

```
#include <semaphore.h>
sem_t sem;

/* Create the semaphore and initialize it to 1 */
sem_init(&sem, 0, 1);
```

Arg 2 (*pshared*): 0 – semafor partajat intre threadurile aceleiasi proces

!= 0 – partajat intre procese si plasat in shared memory

- obtinerea si eliberarea semaforului:

```
/* acquire the semaphore */
sem_wait(&sem);

/* critical section */

/* release the semaphore */
sem_post(&sem);
```

Dealocarea semafoarelor

- semafoare cu nume:

*int sem_unlink(const char *name);*

Obs: semantica de fisiere

- numele semaforului indepartat imediat
- semaforul dispare insa doar dupa ce toate procesele care l-au deschis il inchid cu

*int sem_close(sem_t *sem);*

- semafoare fara nume:

*int sem_destroy(sem_t *sem);*

Semafoare POSIX vs System V

- POSIX create si initializate intr-o singura operatie, System V create intai si apoi initializate independent
- POSIX functioneaza cu increment/decrement 1, System V foloseste valori arbitrare
- POSIX are apeluri non-blocante (tip *trywait* sau timeout), System V foloseste IPC_NOWAIT
- POSIX identifica semafoarele cu/fara nume, System V foloseste chei

Variabile conditie POSIX

- POSIX folosit uzual impreuna cu C/C++
- C/C++ nu ofera suport pentru monitoare la nivel de limbaj

=> POSIX implementeaza monitoare la nivel de biblioteca *threads* prin asocierea variabilelor conditie cu mutex-uri

- creare si initializare “monitor” POSIX:

```
pthread_mutex_t mutex;  
pthread_cond_t cond_var;  
  
pthread_mutex_init(&mutex, NULL);  
pthread_cond_init(&cond_var, NULL);
```


Variabile conditie POSIX (cont.)

- thread care asteapta ca o conditie, **a == b**, sa devina adevarata:

```
pthread_mutex_lock(&mutex);  
while (a != b)  
    pthread_cond_wait(&cond_var, &mutex);  
  
pthread_mutex_unlock(&mutex);
```

- thread care deblocheaza alt thread care asteapta la o variabila conditie:

```
pthread_mutex_lock(&mutex);  
a = b;  
pthread_cond_signal(&cond_var);  
pthread_mutex_unlock(&mutex);
```

Prodicator-consumator POSIX

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 #define ITEMS      8
5 #define BUFFER_SIZE 4
6 int buffer[BUFFER_SIZE];
7
8 void *producer(void*);
9 void *consumer(void*);
10
11 int spaces, items, tail;
12 pthread_cond_t space, item;
13 pthread_mutex_t buffer_mutex;
14
15 int main()
16 {
17     pthread_t producer_thread, consumer_thread;
18     void *thread_return;
19     int result;
20
21     spaces = BUFFER_SIZE;
22     items = 0;
23     tail = 0;
24
25     pthread_mutex_init(&buffer_mutex, NULL);
26     pthread_cond_init(&space, NULL);
27     pthread_cond_init(&item, NULL);
28
29     if(pthread_create(&producer_thread, NULL, producer, NULL) ||
30        pthread_create(&consumer_thread, NULL, consumer, NULL))
31         exit(1);
32
33     if(pthread_join(producer_thread, &thread_return))
34         exit(1);
35     else
36         printf("producer returns with %d\n", (int)thread_return);
37
38     if(pthread_join(consumer_thread, &thread_return))
39         exit(1);
40     else
41         printf("consumer returns with %d\n", (int)thread_return);
42     exit(0);
43 }
44
```

Sisteme de operare

Laborator 10

Semafoare si variabile conditie POSIX

1. Creati un nou subdirector *lab10/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab10/bin*.

2. Rescrieti unul dintre programele C **race.c** din laboratoarele anterioare, fie cel in care doua procese, parinte si copil, scriu concurrent propriul string pe ecran, scrierea fiind facuta caracter cu caracter, fie varianta cu doua thread-uri. Pentru a evidentia mai clar race condition-ul, procesele sau thread-urile repeta operatia de tiparire pe ecran de un numar de ori, sa zicem de 10 ori, in bucla. Folositi un semafor POSIX pentru a impiedica producerea race condition-ului si a obtine un output neintretesut pe ecran.

Ce fel de semafoare POSIX alegeti in functie de varianta de program (cu procese sau cu thread-uri) si de ce?

3. Rescrieti programul C **sync.c** din laboratorul trecut care creeaza de aceasta data doua thread-uri si apoi asteapta terminarea executiei lor. Primul thread doarme pentru o perioada de 5 secunde dupa care afiseaza pe ecran mesajul "*first thread finished\n*" si termina executia. Al doilea thread doarme pentru o perioada de 1 secunda dupa care afiseaza pe ecran mesajul "*second thread finished\n*" si termina executia. Folositi "monitoare" POSIX (mutex-uri si variabile conditie) pentru a va asigura ca indiferent de planificarea thread-urilor, primul thread va tipari intotdeauna mesajul sau inaintea celui de-al doilea.

Daca schimbati valorile de *sleep* intre cele doua thread-uri, primul doarme 1 secunda si cel de-al doilea 5 secunde, mai functioneaza solutia de sincronizare pe care ati implementat-o?

4. Scrieti o biblioteca C **semaphore.c** care implementeaza un semafor numarator (*counting semaphore*) folosind "monitoare" POSIX (mutex-uri si variabile conditie). Semaforul este reprezentat de o structura de date ca mai jos:

```
typedef struct semaphore semaphore_t;

struct semaphore
{
    int counter;
    // completati cu mutex-uri si/sau variabile conditie
    // ...
    void (*init)(semaphore_t *, int );
    int (*down)(semaphore_t *);
    int (*up)(semaphore_t *);
}
```

Definitiiile de mai sus impreuna cu definitiile a doua functii generice *down* si *up* cu semnificatiile de la curs si o functie *init* care initializeaza valoarea unui semafor se stocheaza intr-un fisier **semaphore.h**. Apoi, intr-un program care vrea sa foloseasca implementarea de semafor, variabilele de tip semafor pot fi folosite de de maniera urmatoare:

```
#include "semaphore.h"
```

```
// declaratie semafor
semaphore_t s;

// initializare campuri structura
s.init = init;           # init e functie definita in semaphore.h
                          # si implementata in semaphore.c
s.init(&s, 1);            # semafor binar (mutex)

// folosire semafor
s.down(&s);               # down si up sunt definite in semaphore.h,
s.up(&s);                  # implementate in semaphore.c si setate
                          # corespunzator de init in apelul de mai sus
```

Obs: codul de mai sus presupune ca functia *init* pe care ati scris-o este responsabila nu doar pentru initializarea valorii semaforului, ci si pentru initializarea restului structurii de date (mutex-uri, variabile conditie, *down*, *up*, etc).

Odata implementata functionalitatea ceruta, construiti o biblioteca statica care contine codul compilat al fisierului **semaphore.c**, de exemplu ca mai jos:

```
$ gcc -c semaphore.c -o semaphore.o      # compilati fisierul obiect
$ ar -r libsem.a semaphore.o             # creati o arhiva cu fisierul obiect
$ ranlib libsem.a                        # creati indexul arhivei
```

Apoi rescrieti unul dintre programele **race.c** sau **sync.c** a.i. sa foloseasca implementarea de semafoare din **libsem.a** pentru a rezolva race condition-ul, respectiv problema de sincronizare. De exemplu, dupa ce ati rescris **sync.c** in fisierul **libsem-sync.c** pentru a folosi semafoarele definite in **semaphore.c**, compilati programul astfel:

```
$ gcc -o libsem-sync libsem-sync.c -L. -lsem -lpthread
```

Obs: Comanda de compilare de mai sus presupune ca fisierul **libsem.a** se afla in directorul in care compilati (optiunea -L.). O varianta mai profesionista de a scrie cod este sa creati in *SO/laborator/lab10/* subdirectorul *lib* si sa plasati biblioteca **libsem.a** in *SO/laborator/lab10/lib* dupa care compilarea arata in felul urmator:

```
$ gcc -o libsem-sync libsem-sync.c -L$HOME/SO/laborator/lab10/lib -lsem -lpthread
```

Daca ar fi sa scrieti un Makefile care sa automatizeze intreaga procedura de mai sus, cum ar arata?

5. Scrieti un program C **monitor-shm.c** care foloseste un buffer global ca zona de memorie partajata intre doua thread-uri. Primul thread executa un loop cu 10 iteratii in care scrie succesiv, la fiecare iteratie, urmatorul mesaj in bufferul global (zona de memorie partajata):

```
"this is the first thread printing " + numarul iteratiei
```

La sfarsitul fiecărei iteratii, primul thread asteapta ca al doilea thread sa citeasca mesajul pe care tocmai l-a scris, si abia apoi continua cu urmatoarea iteratie.

Pentru a compune mesajele primul thread poate folosi functia de biblioteca *sprintf*. Mesajele din fiecare iteratie se adauga (*append*) la sfarsitul mesajului scris in iteratia anterioara (cu exceptia primei iteratii).

Al doilea thread executa si el un loop cu 10 iteratii in care, la fiecare iteratie, asteapta ca primul thread sa scrie un mesaj in buffer, citeste mesajul si il afiseaza pe ecran.

Dezvoltati o solutie folosind “monitoare” POSIX (mutex-uri si variabile conditie) care sa permita coordonarea (sincronizarea) accesului celor doua thread-uri la zona de memorie partajata a.i. executia programului sa fie cea anticipata, si anume dupa fiecare mesaj scris de primul thread in memoria partajata (bufferul global), al doilea thread il citeste si il scrie pe ecran. (*Obs: ambele thread-uri executa un loop cu 10 iteratii dar nu puteti face nici o presupunere asupra vitezei relative de executie a celor doua thread-uri*).

6. Scrieti un program C **prod-cons.c** care implementeaza o solutie a problemei producator-consumator folosind doua thread-uri POSIX, unul pentru producator si unul pentru consumator. Pentru implementarea sincronizarii folositi monitoare (mai exact, fiind vorba de POSIX, un mutex si doua variabile conditie, una pentru buffer plin si alta pentru buffer gol).

Care este diferenta in termeni de complexitate a programarii intre solutia cu thread-uri si cea cu semafoare si memorie partajata System V din laboratorul trecut?